OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
TECHNOLOGICAL UNIVERSITY DUBLIN

TECHNOLOGICAL UNIVERSITY DUBLIN
BLANCHARDSTOWN CAMPUS

Academic Term	2019-20
Year of Study	Year 2
Semester	Semester 2
Date of Examination	Summer
Time of Examination	

Programme Code	**Programme Title**	**Module Code**
BN002	Higher Certificate in Science in Computing in information Technology	COMP H2014
BN013	Bachelor of Science in Computing in Information Technology	COMP H2014
BN104	Bachelor of Science (Honours) in Computing	COMP H2014
BN120	Bachelor of Science (Honours) in Computing in Digital Forensics & Cyber Security	COMP H2014

Module Title	Operating Systems

Internal Examiner(s)	**Dr. Kevin Farrell**
External Examiner(s)	**Dr. Christophe Meudec** **Dr. Simon Caton**

Instructions to candidates:

1.	**To ensure that you take the correct examination, please check that the module and programme which you are following is listed in the table above.**
2.	**Answer ANY <u>four</u> questions.**
3.	**Where a question asks you to use diagrams, to obtain full marks, you MUST provide diagrams.**
4.	**Where a question asks you to use code, to obtain full marks, you MUST provide code.**
5.	**This paper is worth 100 marks. All questions are worth 25 marks each.**
6.	**Please make sure you write the question numbers you answer in the boxes on the front of your answer book.**
7.	**This is a two hour examination.**

DO NOT TURN OVER THIS PAGE UNTIL YOU ARE TOLD TO DO SO

Question 1

If you answer this question please write the number on the front of your answer book

a) (i) Describe the *Bounded-Buffer Producer-Consumer Problem*, detailing why a synchronisation solution is necessary.

(4 marks)

(ii) Propose a *shared-memory* solution which does **not** use semaphores. Your answer should include *C* code fragments for the *shared data*, *producer process* and *consumer process*, with an explanation as to how this code operates.

(13 marks)

(iii) Explain any inefficiencies your solution may have.

(2 marks)

b) In relation to process synchronisation, explain what the *critical-section* problem is.

(3 marks)

c) Give the **three** conditions that need to be satisfied in order to solve the *critical-section* problem.

(3 marks)

(25 Marks Total)

Did you answer this question? Have you written the number of the question in the box on the front of your answer book?

Question 2

If you answer this question please write the number on the front of your answer book

a) Describe, with the aid of a diagram, the *Linux Process State Model.* Your diagram must show the correct transitions between *States*. Explain the purpose of each of the *States*.

(12 marks)

b) Using the following headings, describe the **two** modes of execution of processes in an Operating System (OS), namely *user-mode* and *kernel-mode*:

i. What are the features of each mode?
ii. What are the benefits of having the **two** modes?
iii. What is/are the reason/reasons for the OS to switch a process from one mode to the other?
iv. How does the switch between the **two** modes take place; i.e. what is the mechanism?

(8 marks)

c) Briefly explain the role a *Process Control Block* (PCB) plays in a *context switch* in an Operating System. Use diagram(s) as necessary to aid your answer.

(5 marks)

(25 Marks Total)

Did you answer this question? Have you written the number of the question in the box on the front of your answer book?

Question 3

If you answer this question please write the number on the front of your answer book

a) (i) Describe the UNIX/Linux *File Permissions* model. Your answer should include an explanation, with an example, of how the permissions on a file or directory can be represented as a set of *octal* integers.

(8 marks)

b) (i) Consider the extract from a *C* program below, which you studied in the computer laboratory. Explain the sequence of events that occur when the resulting executable program is run. Your answer should indicate which type of process—*parent* or *child*—is executing which line(s) of code and the reason(s) why this is the case.

```
int main(){
   pid_t pid;
   pid = fork();

   if(pid == 0){
     printf("Running xeyes now!\n");
     execlp("/usr/bin/xeyes","Xeyes",NULL);
   }
   else if(pid > 0){
     printf("Hello!\n");
     wait(NULL);
     printf("Goodbye!\n");
   }
   return 0;
}
```

(13 marks)

This Question is continued on the next page...

(ii) In relation to **part b)(i)** above, what happens if the *child process* is killed by the user?

(2 marks)

(iii) In relation to **part b)(i)** above, what happens if, instead, the *parent process* is killed by the user?

(2 marks)

(25 Marks Total)

Did you answer this question? Have you written the number of the question in the box on the front of your answer book?

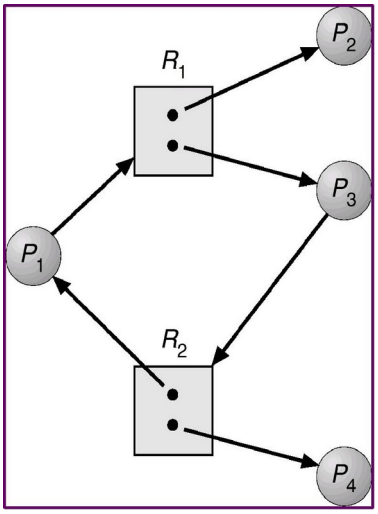
Question 4

If you answer this question please write the number on the front of your answer book

a) Describe the **four** simultaneous conditions necessary in order for *deadlock* to occur.

(4 marks)

b) Explain the following *Resource-Allocation graph*, stating whether or not it indicates the occurance of deadlock. Give reasons for your answer.



(8 marks)

c) In relation to the *Dining Philosophers' Problem* of process/thread synchronisation:

i. Describe the Dining Philosophers' Problem
ii. Explain why a synchronisation solution is necessary.
iii. Describe, with the aid of pseudo-code, one such solution
iv. Outline any advantages/disadvantages your solution may have.

(13 marks)

(25 Marks Total)

Did you answer this question? Have you written the number of the question in the box on the front of your answer book?

Question 5

If you answer this question please write the number on the front of your answer book

a) Distinguish between *soft real-time* operating systems and *hard real-time* Operating Systems.

(4 marks)

b) (i) Describe the *First-Come First-Served* scheduling algorithm by addressing the following questions:

- Is it Pre-emptive or Non-pre-emptive?
- What are the uses of this algorithm?
- What are the disadvantages of this algorithm?

(3 marks)

(ii) The following table contains data concerning **five** different processes when the *First-Come First-Served* scheduling algorithm is used (all processes are assumed to arrive at time 0 in the order: Process #1, #2, #3, #4, #5):

Process #	***Estimated Run Time***	***Waiting Time***
1	2	0
2	60	2
3	1	62
4	3	63
5	50	66

Using the same data, compile a similar table for the *Shortest-Job First* scheduling algorithm.

(5 marks)

(iii) Using the *average waiting-time* as a metric for comparing how efficient the algorithms in **part b)(ii)** are, which is better? Give reasons for your answer.

(8 marks)

This Question is continued on the next page...

c) In a particular program, the serial fraction of code is **5%** (i.e. **5%** of the code is not parallelisable). According to *Amdahl's Law,* what is the *speed-up* achievable for this program, run on a multi-core system with **32** cores? What is the theoretical maximum speed-up achievable with **32** cores (i.e. assuming no serial fraction)?

Amdahl's Law:

$$\text{Speed-Up} \leqslant \frac{1}{S + \frac{(1-S)}{N}}$$

where:

- S = Serial Fraction of Code
- N = Number of CPU Cores

(5 marks)

(25 Marks Total)

Did you answer this question? Have you written the number of the question in the box on the front of your answer book?